# Como medir os seus agentes de IA

Só para o palestrante: o que dizer em cada slide, transições, termos para explicar, a revelação em três versões, plano B, checklist e perguntas prováveis.

---

1. Guia de fala

Fonte: https://github.com/MateusCastro2203/agent-evaluation

# Guia de fala

Este guia é material do palestrante (também fica no repo, para quem quiser reapresentar a palestra); não entra no ebook. Ele segue os 24 slides (23 mais o apêndice de referências) de `palestra/slides.md`, na mesma ordem e com os mesmos tempos das notas do apresentador. Para cada slide: o que dizer (em frases curtas), a transição para o próximo e os termos que valem uma frase de explicação.

**Para quem é a palestra.** A maioria da sala é iniciante. Fale como numa conversa: frases curtas, uma ideia por frase, e todo termo técnico explicado na mesma frase em que aparece ("a margem de erro, que no jargão se chama intervalo de confiança"). Os detalhes técnicos estão nas notas e nas respostas às perguntas, para quem pedir.

> **Se só der para lembrar de uma coisa:** "parece que melhorou" não é medida. Dá para medir o seu agente com tarefas e um teste automático, repetindo várias vezes, e comparar o antes e o depois de uma mudança. Isso é uma eval. A palestra não depende do resultado do run: qualquer que seja o veredito, a lição é a mesma, e o relatório diz quantas execuções faltam para ter certeza.

**Valores do run** (de `evals/promptfooconfig.yaml` e `resultados/ensaio/NOTAS.md`): 2 variantes × 8 tarefas × 2 repetições = 32 execuções, modelo `sonnet`, concorrência 4. No ensaio, o run inteiro levou **~3 min 28 s** de relógio (cada execução, de 12 s a 42 s). O run dispara no minuto 1 e termina muito antes da revelação. Quando termina, o próprio `pnpm eval` gera o relatório (`resultados/relatorio.html`), e o slide 18 mostra esse arquivo sozinho. Se você mudar repetições, concorrência ou modelo, rode o ensaio de novo e atualize os números deste guia.

**Resultado do ensaio** (use como exemplo, nunca como "o resultado de hoje"): A 12/16 (75%, margem de erro de 51% a 90%), B 16/16 (100%, de 81% a 100%), "resolve sempre" (pass^2) 6/8 × 8/8, 3 "mexeu onde não devia" (fora do escopo) e 1 teste escondido falhando, todos de A (o teste na tarefa 04), 0 erro do agente. Veredito: **inconclusivo** ("ainda não dá para saber"); o relatório estima 27 execuções de cada lado para as margens pararem de se cruzar (critério do veredito), com piso de 24 pelo teste z de duas proporções.

---

# Antes de subir no palco

## Na véspera

1. **Não regravar o ensaio.** Ele é o run completo de 2026-10-27 (A 12/16 × B 16/16, inconclusivo, 27 por variante, 3 min 28 s), e esses números estão nos slides 18 e 22, no README e no ebook. Se regravar, atualize todos esses lugares. **Cuidado:** `pnpm ensaio:gravar` copia `resultados/ultimo.json`, que qualquer `pnpm eval` (inclusive o `pnpm eval:rapido`) sobrescreve.
2. **Como o `pnpm eval` termina.** Ele roda o promptfoo e, em seguida, o relatório. O promptfoo sai com código 100 quando alguma execução não passou; isso é normal e não impede o relatório. O código final do `pnpm eval` é o do relatório: 0 em "ainda não dá para saber" e "B melhor", 1 só em "A melhor" (o portão). Não se assuste se aparecer um erro do pnpm nesse caso.
3. Rodar `pnpm relatorio:ensaio` e conferir que ele regenera `resultados/ensaio/relatorio.html`. Ele só lê e escreve arquivo (sem rede, sem modelo), então funciona offline. É desse arquivo que o plano B copia.
4. Rodar `pnpm relatorio:planob`, abrir `palestra/apresentacao.html`, ir ao slide 18 e conferir que o relatório do ensaio aparece dentro do slide. Depois, `pnpm relatorio:aguardando` e conferir que o slide 18 mostra "O agente ainda está trabalhando…".
5. Ler o relatório do ensaio de cima para baixo e decorar três coisas: o título ("Ainda não dá para saber"), as duas linhas ("12 de 16" × "16 de 16") e o "seriam umas 27 execuções de cada". Se o run ao vivo falhar, é isso que você vai mostrar.
6. Conferir que a página de links (o QR dos slides 2 e 23) já aponta para o repositório publicado e para o ebook.

## Checklist pré-palco (30 minutos antes)

- ☐ **`claude` logado:** abrir `claude` no terminal, na pasta do repo, e confirmar que ele entra direto, sem pedir login. O repo usa o login normal do Claude Code; não há chave de API em lugar nenhum.
- ☐ **`pnpm eval:rapido` OK no local:** roda 1 tarefa, 1 repetição (e não gera relatório). Se passar, rede, login e cota estão funcionando. Se falhar, você já sabe que vai de plano B.
- ☐ **`pnpm relatorio:aguardando` por último**, depois do `eval:rapido`: grava a página "O agente ainda está trabalhando…" em `resultados/relatorio.html`. Sem isso, o slide 18 pode mostrar um relatório velho.
- ☐ **Notebook na tomada.** O run usa CPU e rede por alguns minutos (no ensaio, ~3 min), e o notebook fica ligado a palestra inteira.
- ☐ **Sem modo de suspensão:** desligar a suspensão e o descanso de tela nas configurações; no macOS, deixar `caffeinate -dims` rodando num terminal à parte durante a palestra.

- [ ] **Terminal com fonte grande** e na pasta do repo, com `pnpm eval` já digitado, sem dar Enter. Um segundo terminal, também na pasta do repo, para o plano B (`pnpm relatorio:planob`).
- [ ] **`resultados/ensaio/resultado.json` é o run completo do ensaio (2026-10-27),** não o do `pnpm eval:rapido` de agora há pouco (o rápido só sobrescreve `resultados/ultimo.json`).
- [ ] **Reserva extra (opcional):** uma aba do navegador com `resultados/ensaio/relatorio.html`, caso o slide 18 não carregue o relatório por algum motivo.
- [ ] **Notificações desligadas** (modo Não Perturbe) e Slack/e-mail fechados.
- [ ] **Apresentar pelo `palestra/apresentacao.html`** aberto no navegador (Chrome ou Edge), em tela cheia (tecla `f`). Seta para a direita ou espaço avançam **cada clique** (frases de efeito, linhas de lista) e depois o slide; seta para a esquerda volta. A tecla `p` abre a visão do apresentador numa janela à parte, com as notas e o próximo slide. O HTML funciona sem internet.
- [ ] **Passar os slides uma vez no HTML** antes de subir, para conferir que os cliques aparecem na ordem (slides 6, 13, 14, 15, 17; as perguntas não têm cliques) e que o slide 18 mostra a página de espera.
- [ ] **PDF como reserva:** `palestra/apresentacao.pdf` aberto noutra janela. Ele mostra cada slide já completo (sem cliques) e, no slide 18, só um aviso no lugar do relatório.
- [ ] Água por perto. São 40 minutos no total, sem pausa: ~31 min de palestra com o projeto ao vivo e ~9 min de perguntas.

---

## Slide a slide

**Como funcionam as perguntas.** Os slides com um selo laranja em cima ("Mão para cima", "Façam a aposta", "Chute antes de eu mostrar") são perguntas para a plateia, respondidas com a mão ou em voz alta. Regra para todas: o slide de pergunta é **um slide só, sem cliques**: tudo o que aparece já está na tela. Faça a pergunta, espere no máximo 20 segundos e conte as mãos por alto ("uns dois terços", "quase ninguém"). Nos slides "Chute antes de eu mostrar", a resposta fica no **slide seguinte**: só avance depois de ouvir a sala. O objetivo é a sala acordar, não fazer pesquisa.

**Como funcionam os cliques.** No `apresentacao.html`, seta para a direita ou espaço mostram o próximo item do slide e, quando não há mais itens, passam para o próximo slide. Cada slide diz abaixo quantos cliques tem. Slides de pergunta e de resposta não têm cliques.

---

### Slide 1 — Capa · [00:00–00:15]

**Diga:** *"Hoje a pergunta é: como saber se o seu agente de código está bom?"* e avance.

---

## Slide 2 — Quem é Mateus Castro? · [00:15–00:45]

**Diga, em 30 s no máximo:** uma frase por bloco do slide. Onde trabalha e com o quê ("AI Engineer na Creditas, com foco em soluções de IA"), por onde já passou (Gol, Smiles, Neon, Dotz) e quem é fora do código.

**Sobre o QR:** *"Esse QR leva para a minha página de links: repositório, ebook e slides. Não precisa fotografar agora: ele volta no último slide."*

Não se demore aqui: o run dispara no slide 4.

---

## Slide 3 — Pergunta: quem mede? · [00:45–01:15] · sem cliques

**Pergunte:** *"Quem aqui mede hoje se o agente de código está melhorando?"*

**Leia a sala, opção por opção:** *"Quem não mede?"* (mãos) → *"Quem vai no feeling?"* (mãos) → *"Quem tem uma métrica?"* (mãos). Levante a mão junto no "no feeling": você também já fez. Até 20 s no total.

**O que dizer em cada caso:**

- **Quase ninguém tem métrica** (o esperado): *"Normal. É para isso que a gente está aqui."*
- **Muita gente tem métrica:** *"Ótimo. No fim vocês me dizem se a minha é melhor que a de vocês."*
- **Ninguém levanta a mão:** *"Tudo bem, a próxima vale aposta."*

**Transição:** *"E para não ficar só na teoria, eu vou fazer uma aposta agora."*

---

## Slide 4 — A aposta de hoje · [01:15–02:15]

**Faça:** dar Enter no `pnpm eval` que já está digitado. Confirmar que começou (aparece o progresso do promptfoo) e minimizar o terminal. Não precisa voltar nele: quando o run termina, o `pnpm eval` gera o relatório sozinho, e o slide 18 mostra.

**Diga:** *"Eu escrevi um `CLAUDE.md` para este projeto. É um arquivo de texto com as regras da casa, que o agente lê antes de começar. Acho que ele ajuda. Mas 'acho' não é medida. Então o agente vai fazer 8 tarefas, duas vezes cada, sem o arquivo e com o arquivo. São 32 execuções, rodando agora, enquanto eu falo. No fim, a gente vê."*

**Termos:**

- **`CLAUDE.md`:** um arquivo de texto na raiz do projeto que o Claude Code lê no começo de toda conversa. É onde você escreve as regras da casa: como rodar o teste, os padrões, as pastas que não pode mexer.

- **Variante:** cada uma das versões que você compara. Aqui só muda uma coisa entre A e B: ter ou não o `CLAUDE.md`.

**Se o `pnpm eval` não começar** (erro de login ou rede): não tente consertar no palco. Diga *"o agente decidiu participar do plano B hoje; no fim eu mostro o run que gravei no ensaio, com o mesmo comando"* e siga.

---

## Slide 5 — Pergunta: a aposta · [02:15–03:00] · sem cliques

**Pergunte:** *"O `CLAUDE.md` vai fazer diferença?"*

**Leia a sala, opção por opção:** *"Quem acha que o B, com o arquivo, ganha?"* → *"Quem acha que dá empate?"* → *"Quem acha que o A, sem o arquivo, ganha?"*. Conte por alto e **guarde a proporção** (ex.: "uns 70% no B, uns 20% no empate, quase ninguém no A"): você volta nela no slide 18. Até 20 s.

**O que dizer em cada caso:**

- **Quase todo mundo no B:** *"Vocês confiam no meu arquivo. Vamos ver se ele merece."*
- **Muita gente no empate:** *"Tem gente desconfiada. Bom sinal."*
- **Ninguém no A:** *"Ninguém no A? Guardem isso."*

**Fechamento:** *"A resposta sai no fim da palestra, no relatório ao vivo."*

**Transição:** *"Enquanto ele trabalha, vamos ao problema."*

---

## Slide 6 — "Parece que melhorou" · [03:00–04:45] · 1 clique

**Diga (apontando o fluxo):** *"Todo mundo já fez isso. Mudou alguma coisa. Pediu uma tarefa. Deu certo. 'Melhorou!' Mas a tarefa era fácil? E se pedisse de novo? Quanto custou? Mexeu onde não devia?"* Clique: *"E a mais esquecida: daria certo antes da mudança também?"*

**Pergunta opcional** (só se a sala estiver fria): *"Quem já trocou de modelo porque 'sentiu' que estava melhor?"* Levante a mão junto.

O tom não é de bronca. Testar no olho é o ponto de partida. O problema é parar nele.

**Diga (transição):** *"O agente não dá sempre a mesma resposta. Mesmo pedido, mesmo código, e ele faz caminhos diferentes. Isso é normal. Então uma execução é como jogar uma moeda uma vez: não diz se a moeda é viciada."*

**Termos:**

- **Vibe check:** avaliar "no olho", pela sensação de uso, sem número.
- **Não determinístico:** a mesma entrada pode dar saídas diferentes. Agente é assim.

**Transição:** *"Então quanto vale uma execução? Chutem."*

---

## Slide 7 — Chute: "1 de 1 é 100%. Confiam?" · [04:45–05:15] · sem cliques

**Pergunte:** *"Passou 1 de 1. É 100%. Quem confia?"* Mãos; até 15 s.

**O que dizer em cada caso:**

- **Muita gente confia:** *"Vamos ver o que a conta diz."*
- **Ninguém confia:** *"Desconfiança saudável. Quanto, então?"* (deixe alguém gritar um número).

**Transição:** *"Vamos ver o que a conta diz."*

---

## Slide 8 — Resposta: 1 de 1 · [05:15–07:00] · sem cliques

**Diga:** *"Pensem na pesquisa eleitoral. Ela sempre vem com uma margem de erro. Aqui é igual. Com 1 de 1, a taxa real pode estar em qualquer lugar entre 21% e 100%. Com 16 de 16, a faixa fica bem menor: de 81% a 100%. Mesmo 16 de 16 não prova 100%."*

**Diga:** *"Isso é conta, não opinião. E é a mesma conta que o relatório vai usar no fim."*

**Termo:**

- **Margem de erro** (no jargão, intervalo de confiança de 95%; o cálculo usado é o intervalo de Wilson): a faixa onde a taxa verdadeira provavelmente está, dado o que você viu. Quanto mais execuções, mais estreita.

**Transição:** *"Se uma execução não basta, o que eu meço, e quantas vezes?"*

---

## Slide 9 — As métricas de um agente de código · [07:00–09:00]

Vá linha por linha, devagar. É o slide mais "de levar para casa" do bloco.

**Diga:**

1. *"Sucesso: deu certo? Aqui, 'deu certo' quer dizer que um teste automático passou."*
2. *"Custo, voltas e tempo: quanto ele gastou, quantas idas e vindas deu com as ferramentas, quanto demorou. O próprio Claude Code devolve esses números."*
3. *"Escopo: mexeu onde não devia? Fica separado de propósito. Resolver mexendo onde não devia não é sucesso."*
4. *"Consistência: resolve sempre? Acertar uma vez pode ser sorte. Hoje cada tarefa roda duas vezes, e a gente conta quantas passaram nas duas."*

**Sobre custo:** *"O valor em dólar é uma estimativa do próprio Claude Code. Com assinatura, leia como 'tamanho do esforço', não como conta a pagar."*

**Termos:**

- **Token:** um pedacinho de texto que o modelo lê ou escreve (mais ou menos uma sílaba ou palavra curta). Mais tokens, mais trabalho, mais custo.
- **Turno (volta):** uma ida e volta entre o modelo e uma ferramenta: ler um arquivo, editar, rodar o teste.
- **pass^k:** o nome técnico do "resolve sempre?": passou em todas as k tentativas.
- **Critério de feito:** a regra objetiva que diz se a tarefa foi cumprida. Aqui, um teste automático.

**Transição:** *"Métrica definida. Agora: como montar a prova."*

---

## Slide 10 — O conjunto de hoje: 8 tarefas · [09:00–11:00]

**Diga:** *"O app de teste é uma agenda de meetups, inventada. As tarefas estão escritas como um pedido de verdade: 'não deixe inscrever em evento lotado', e só. Cinco tarefas dependem de uma regra da casa que só o `CLAUDE.md` explica. Por exemplo: dinheiro sempre em centavos, ou uma pasta antiga que ninguém pode mexer. As outras três são de controle: não dependem de regra nenhuma."*

**Por que ter controle:** *"Se o B ganhar também nas de controle, desconfie. O arquivo não deveria mudar nada ali. O efeito esperado está nas tarefas 03 a 07."*

**Por que o pedido não cita a regra:** se dissesse "use o erro do projeto", A e B empatariam e a gente não mediria nada.

**Termos:**

- **Eval** (de *evaluation*, avaliação): um teste automático para um sistema com IA. Uma lista de tarefas com um jeito automático de dizer se deu certo; o sistema roda em todas e você conta quantas acertou.
- **Tarefa de controle:** caso em que a mudança não deveria fazer diferença. Serve de régua.
- **Convenção (regra da casa):** regra do projeto que não está escrita no código. Um dev novo só descobre lendo a documentação ou levando um comentário no code review.

**Transição:** *"Vamos abrir uma delas."*

---

## Slide 11 — Uma tarefa por dentro · [11:00–13:00]

**Diga:** *"Em cima, o pedido da tarefa 03. É só isso que o agente vê. Embaixo, o que o teste escondido confere. Não basta dar erro. Tem que ser o erro do projeto, com o código em maiúsculas, falando de lotação. E não pode reaproveitar um código antigo que significa outra coisa. É exatamente o que um*

*dev novo erraria sem ler a documentação."*

**Analogia:** *"O teste escondido é a prova que o aluno não viu. Ele estuda pelo enunciado, e a correção usa um gabarito que fica com o professor."*

**Se perguntarem "por que não exigir um código exato?":** *"Porque o pedido não diz qual código usar. Exigir um nome letra por letra mediria sorte. O teste aceita qualquer nome razoável de lotação e só recusa o código antigo,* `CAPACIDADE_INVALIDA` *."*

**Se perguntarem "o agente não pode trapacear?":** *"O teste nunca entra na pasta do agente. A correção roda numa pasta nova, com o app intacto e só os arquivos que ele mudou. O teste escondido entra por último, por cima de qualquer arquivo com o mesmo nome."*

**Termos:**

- **Teste escondido (oculto):** o teste que decide se a tarefa passou, guardado fora do alcance do agente.
- **Escopo:** a lista de arquivos que a tarefa permite alterar.
- **Código em maiúsculas (SCREAMING_SNAKE):** tudo maiúsculo, com sublinhado entre as palavras (ex.: `EVENTO_LOTADO`).

**Transição:** *"E como uma execução dessas roda, do começo ao fim?"*

---

## Slide 12 — Como uma execução roda · [13:00–14:30]

**Diga:** *"Três passos. Um: o agente trabalha numa cópia do app, nunca no código de verdade. Dois: ele faz a tarefa trancado numa caixa, sem internet, só com as ferramentas que eu liberei. Três: um teste que ele não viu confere o trabalho, e eu vejo se ele mexeu só nos arquivos permitidos."*

**Diga:** *"Quem organiza as 32 execuções é o promptfoo, uma ferramenta gratuita de evals. Ele chama o agente, repete e grava tudo num arquivo. A conta do resultado fica num script do repo."*

**Isolamento (fale só se houver gente mais experiente perguntando):** *"A cópia não recebe as minhas configurações pessoais nem os meus conectores. Se recebesse, a variante A teria um* `CLAUDE.md` *escondido, e o experimento estaria contaminado. Eu conferi: perguntei para a variante A que instruções ela recebeu, e ela respondeu 'nenhuma'."* O que **não** dá para tirar são as configurações gerenciadas pela organização; elas chegam às duas variantes, igualmente.

**As flags, se perguntarem** (de `src/claude.ts`): `-p <prompt> --output-format json --model sonnet --max-turns 30`; `--permission-mode dontAsk` com `--allowedTools` (ler e editar só dentro da cópia, `pnpm test`, `pnpm vitest` e comandos de leitura: ls, cat, grep, head, wc); `--disallowedTools` (WebFetch, WebSearch, as deps do lab, o projeto e `~/.claude`); sandbox do SO por `--settings`; `--setting-sources project`; `--strict-mcp-config`; `--no-session-persistence`. Timeout de 10 minutos por execução.

**Login:** *"Usa a assinatura normal do Claude Code. Não precisa de chave de API."*

**Termos:**

- **Headless (`claude -p`):** o agente rodando sem tela interativa. Recebe o pedido e devolve o resultado num arquivo.
- **Sandbox:** uma "caixa" do sistema operacional que limita o que o programa pode ler, escrever e acessar na rede.
- **promptfoo:** ferramenta gratuita (open source) para rodar evals a partir de um arquivo de configuração.
- **Provider:** no promptfoo, o pedaço de código que executa cada caso. Aqui, roda o Claude Code numa cópia do repo.
- **Diff:** a lista do que mudou nos arquivos em relação à cópia original.
- **MCP:** o jeito padrão de ligar o agente a ferramentas externas (Slack, e-mail, banco). Aqui fica desligado.

**Transição:** *"E quando não dá para conferir com teste?"*

---

## Slide 13 — IA como juiz · [14:30–16:00] · 1 clique

**Diga:** *"Às vezes não dá para escrever um teste. 'A documentação ficou clara?' Aí dá para pedir para outra IA dar nota. É o que o mercado chama de LLM-as-judge. Ajuda no que teste não pega: clareza, estilo. Mas o juiz também varia. É como um corretor de redação que, no mesmo texto, às vezes dá 7 e às vezes dá 9. Juiz que varia avaliando agente que varia: a incerteza soma."* Clique: *"Se dá para conferir com código, confira com código."* Hoje nenhuma tarefa precisa de juiz: tudo é teste.

**Pergunte (opcional, se o tempo deixar):** *"Que tarefa do seu dia a dia não dá para conferir com teste?"* Uma ou duas respostas, até 20 s. Respostas esperadas: "a documentação ficou clara?", "o PR está bem descrito?". Resposta: *"Ali o juiz pode valer. Mas meça o juiz também: dê a sua nota numa amostra e veja se ele concorda com você."*

**Do caso real (uma frase):** *"No caso real que eu vou contar agora, o juiz foi rejeitado por isso: juiz variável avaliando agente variável."*

**Termos:**

- **LLM-as-judge (IA como juiz):** um modelo dando nota ao trabalho de outro.
- **Viés:** tendência do juiz, como preferir a resposta mais longa ou a que aparece primeiro.

**Transição:** *"Isso não é teoria de laboratório. Vou mostrar o que a gente aprendeu fazendo isso em produção."*

---

## Slide 14 — O que conta como "deu certo" · [16:00–17:30] · 1 clique

**Contexto (diga antes):** *"Um time da Creditas construiu agentes que resolvem chamados de suporte operacional, o trabalho de 'manter as luzes acesas'. Não vou mostrar números internos nem nomes de sistemas: só o que aprendemos sobre medir."*

**Cuidado durante todo o bloco:** não citar números, nomes de pessoas, nomes de sistemas ou parceiros, chaves de ticket nem achados de segurança, nem de memória, nem "por alto". Se perguntarem, use a resposta da pergunta 7 da seção de perguntas.

**Diga (lição 1):** *"Toda execução termina com um rótulo: resolvido, assistido, escalado ou falta informação. 'Funcionou?' é pergunta vaga demais."*

**Clique (lição 2):** *"Agora imaginem: entra um caso que nem era para o agente resolver, e ele conta como falha. A taxa cai. Ninguém sabe se o agente piorou ou se só chegou outro tipo de caso. Por isso: decida o que é 'dar certo' antes de olhar o número."*

**Ligação com o repo:** *"No repo de hoje são quatro rótulos: passou, falhou, mexeu onde não devia e erro do agente. Erro do agente, tipo cair a internet, fica à parte."*

**Termos:**

- **KTLO** (*keep the lights on*, "manter as luzes acesas"): manter funcionando o que já existe (chamados, correções, rotinas).
- **Assistido:** o agente diagnosticou, mas uma pessoa executou. Contar como "resolvido" infla o número; contar como "falha" esconde valor.
- **Escalado:** o agente passou o caso para uma pessoa.

**Transição:** *"E a eval precisa ficar melhor com o tempo."*

---

## Slide 15 — Eval que só melhora · [17:30–19:00] · 3 cliques (um por linha)

**Diga (linha 1):** *"A resposta certa de cada caso, o gabarito, é escrita por uma pessoa, do zero, sem olhar o que o agente respondeu. Se você copia a resposta do agente para o gabarito, está medindo o agente contra ele mesmo."*

**Diga (linha 2):** *"A gente mantém uma lista dos tipos de caso que ainda não têm teste. Essa lista só pode diminuir. Se alguém tirar um teste, o build quebra. É uma catraca: só gira para um lado."*

**Diga (linha 3):** *"Falhou na terça, teste na quarta. Todo erro do dia a dia vira uma tarefa nova com teste escondido. E um aprendizado só vira regra quando aparece em dois ou mais casos, para não transformar uma exceção em regra para todo mundo."*

**Termos:**

- **Gabarito:** o resultado esperado de cada caso, escrito por uma pessoa.
- **Cobertura:** ter pelo menos um teste para aquele tipo de caso.

- **Catraca:** mecanismo que só gira para um lado. Aqui: a cobertura só cresce.

**Transição:** *"Então, como eu uso tudo isso para melhorar o meu agente?"*

---

## Slide 16 — Uma mudança por vez · [19:00–20:30]

**Diga:** *"Quatro passos. Roda a eval antes, para ter o 'antes'. Muda uma coisa só. Roda de novo, várias vezes. Compara olhando a margem de erro, não só a média. É como testar uma receita: se você troca o forno e a farinha no mesmo dia e o bolo melhora, não sabe qual dos dois ajudou."*

**Diga:** *"'Uma coisa' pode ser o `CLAUDE.md`, uma skill, um hook, o modelo ou o pedido. Hoje eu mudei só a existência do `CLAUDE.md`."*

**Dica prática:** *"Guarde a configuração do agente junto com o resultado da eval. Daqui a um mês você sabe qual versão deu qual número."*

**Termos:**

- **Linha de base (baseline):** a medida de antes da mudança, para comparar.
- **Teste A/B:** comparar duas versões que diferem em uma coisa só. O experimento de hoje é um.

**Transição:** *"E onde mora cada tipo de regra?"*

---

## Slide 17 — Onde cada regra mora · [20:30–22:00] · 1 clique

**Diga, uma linha de cada:**

- *"`CLAUDE.md`: um texto com as regras do projeto, que o agente lê no começo de toda conversa. Bom para convenção, comando de teste, pasta proibida. Mas é um conselho, não uma trava: a própria documentação diz isso."*
- *"Skill: um manual que o agente só abre quando a tarefa pede. Bom para um procedimento longo. Enquanto não é usado, quase não custa nada. Se a descrição for ruim, ele nunca abre."*
- *"Hook: um script seu que roda sozinho antes ou depois de cada ação do agente, e pode bloquear a ação. É para a regra que precisa valer sempre. É código, então se testa como código."*

**Clique:** *"Regra que não pode falhar não mora no texto. Mora no hook, no teste ou no CI."*

**Ligação com o repo:** *"Cada uma dessas é uma variante B possível: troque o `CLAUDE.md` por uma skill ou um hook e rode de novo."*

**Termos:**

- **Contexto:** tudo o que o modelo lê antes de responder. Influencia, mas não obriga.
- **CI:** a esteira que roda os testes automaticamente a cada mudança no repositório.

**Transição:** *"Já deu tempo. Vamos ver o que o agente fez."* E avance: o slide 18 mostra o relatório sozinho. Não precisa ir ao terminal nem ao navegador.

---

## Slide 18 — Ao vivo: o relatório (com a volta da aposta) · [22:00–25:00]

**Primeiro, a volta da aposta do slide 5:** *"No começo, [a proporção que você guardou, ex.: uns 70%] apostou no B, [x] no empate e [quase ninguém] no A. Vamos ver."* Não peça mãos de novo.

**Como o slide funciona:** ele recarrega `resultados/relatorio.html` toda vez que aparece. Quando o run termina, o `pnpm eval` grava ali o relatório. Antes disso, está ali a página de espera que você gravou com `pnpm relatorio:aguardando`.

**O que fazer, conforme o que aparece:**

- **Apareceu o relatório:** confira, nas duas linhas grandes, que diz "de 16". Se aparecer outro número (por exemplo "de 1", de um `eval:rapido` antigo, ou bem menos de 16, porque o run caiu no meio), não leia: vá para o plano B.
- **Aparece "O agente ainda está trabalhando…":** fale 1 ou 2 minutos (o exemplo do ensaio, ou a regra "faixas que se cruzam = ainda não dá para saber"), volte um slide e avance de novo. Se não sair, plano B.
- **O run caiu:** plano B (seção abaixo). Não espere no palco.

**Diga:** *"Vou ler de cima para baixo, na ordem da tela."*

1. **O título:** *"Ainda não dá para saber"*, *"Com CLAUDE.md foi melhor"* ou *"Sem CLAUDE.md foi melhor"*. Compare com a aposta da plateia: *"quem apostou no [B / empate / A]..."*. Se a maioria apostou em empate e deu "ainda não dá para saber", diga: *"quase: não é 'são iguais', é 'não sei ainda'."*
2. **As duas linhas grandes:** *"Sem o arquivo, 12 de 16 deram certo. Com o arquivo, 16 de 16."* (com os números de hoje).
3. **As faixas coloridas:** *"O tracinho é o resultado de hoje. A faixa é a margem de erro, como na pesquisa eleitoral. Se as faixas se cruzam, ainda não dá para dizer quem é melhor."* A frase do topo diz quantas execuções de cada seriam precisas.
4. **Na tabela, se der tempo:** *"Resolve sempre?"* (quantas tarefas passaram nas duas tentativas) e *"Mexeu onde não devia"*.
5. **Tarefa por tarefa (rolar o relatório só se der tempo):** *"O ganho, se houver, está nas tarefas de regra da casa (03 a 07) ou nas de controle?"*

**O portão (uma frase):** *"No repo, isso também roda como portão: só barra a mudança se o A for melhor que o B. 'Ainda não dá para saber' passa."*

**Não comemore nem lamente ainda:** *"Os próximos slides explicam como ler isso sem se enganar."* A fala da revelação, já adaptada ao resultado, está na seção "A revelação em 3 versões". Escolha a versão aqui, olhando o título.

---

## Slide 19 — Chute: "B acertou mais. Então B ganhou?" · [25:00–25:30] · sem cliques

A pergunta é sobre o relatório que a plateia acabou de ver no slide 18.

**Pergunte:** *"B acertou mais que A. Quem acha que B ganhou?"* Mãos; até 15 s.

**O que dizer em cada caso:**

- **A maioria disse "ganhou":** *"É a resposta natural. E é exatamente o 'parece que melhorou' do começo."*
- **A maioria disse "não":** *"Vocês já pegaram o jeito. Agora a regra."*

**Se hoje B não acertou mais** (empate ou A na frente): pergunte *"Então A e B são iguais?"*. A resposta do slide seguinte vale do mesmo jeito.

**Transição:** *"Vamos ver como se lê isso."*

---

## Slide 20 — Resposta: quando dá para dizer quem ganhou · [25:30–26:45] · sem cliques

O slide não tem números de propósito: é a regra para ler qualquer relatório, inclusive o de hoje.

**Diga:** *"Cada variante tem a sua faixa: a margem de erro. Se as faixas se cruzam, como na esquerda, ainda não dá para saber quem é melhor. Só quando elas se separam, como na direita, dá para dizer."* Depois, volte ao relatório de hoje com a fala: *"No nosso, as faixas se cruzam. Então: ainda não dá para saber."* (Se hoje elas se separaram, diga: *"No nosso, elas se separaram: dessa vez dá para dizer."*)

**Analogia:** *"É a pesquisa eleitoral com poucas entrevistas. Dois candidatos parecem longe, mas as margens se cruzam: é empate técnico. Ouvindo mais gente, a margem diminui."*

**Diga a regra do repo:** *"Se a faixa de B fica toda acima da de A, B é melhor. Se fica toda abaixo, A é melhor. Se elas se cruzam, ainda não dá para saber. E isso não é empate: é 'não tenho dado para dizer'."*

**Se alguém mais experiente perguntar "por que não um teste de hipótese?":** *"Um teste z de duas proporções declararia vencedor mais cedo. Exigir faixas sem cruzar é mais rigoroso, de propósito: o repo prefere não declarar vencedor cedo. O relatório mostra os dois números."*

**Por que Wilson e não a conta simples (`p ± 1,96·√(p(1−p)/n)`), se perguntarem:** com poucas execuções ou taxa perto de 0% ou 100%, a conta simples dá faixas absurdas (16 de 16 daria "100% ± 0"). O intervalo de Wilson não tem esse problema.

**Termos:**

- **Intervalo de Wilson:** o jeito de calcular a margem de erro que o repo usa; funciona bem com poucas execuções. Proposto por Wilson em 1927.
- **Inconclusivo:** o nome técnico de "ainda não dá para saber".

**Transição:** *"Então, quantas execuções eu precisaria?"*

---

## Slide 21 — Chute: quantas execuções? · [26:45–27:15] · sem cliques

**Pergunte:** *"Quantas execuções de cada lado para ter certeza? Quem diz 5? 16? 40? 200?"* Mãos em cada uma; até 20 s no total.

---

## Slide 22 — Resposta: 27 de cada lado · [27:15–29:00] · sem cliques

**Diga:** *"No ensaio, com A 12 de 16 e B 16 de 16, o relatório diz 27 de cada lado."* Diga **"do ensaio"** em voz alta: é o run gravado, não o de hoje. Se o relatório de hoje mostrou outro número na frase do topo ("seriam umas [N] execuções de cada"), complete: *"no de hoje, deu [N]"*. Se as duas taxas forem iguais, o relatório diz que não dá para calcular.

**O que dizer em cada caso:**

- **Muita gente no 16:** *"Chegaram perto, mas é o que a gente rodou hoje, e não bastou."*
- **Muita gente no 40:** *"Um pouco a mais do que precisava."*
- **Muita gente no 200:** *"Para essa diferença, menos. Mas o 200 não é absurdo: com uma diferença pequena, chega lá."*

**Analogia:** *"Jogar uma moeda 4 vezes e dar 3 caras não prova que ela é viciada. Jogar 400 vezes e dar 300, prova. Quanto menor a diferença que você quer enxergar, mais vezes precisa jogar: 62,5% contra 75% pediria 212 execuções de cada lado."*

**Se perguntarem de onde vem o número:** o relatório testa 1, 2, 3… execuções com as taxas de hoje e para na primeira em que as faixas deixam de se cruzar. Nos detalhes técnicos do relatório também aparece um "piso" pela fórmula clássica (teste z de duas proporções, 95% de confiança, 80% de poder), que costuma ser menor; no ensaio, 27 com piso de 24. A fórmula do piso: `n ≈ (1,96 + 0,84)² · [pA(1−pA) + pB(1−pB)] / (pA − pB)²`.

**Diga (fechamento, vale para qualquer resultado):** *"Eu não sei ainda se o meu `CLAUDE.md` presta. Mas agora eu sei quanto eu preciso rodar para saber. Isso é medir."* (Na versão "B ganha", adapte: veja a seção da revelação.)

**Termos:**

- **Poder (estatístico):** a chance de o experimento enxergar a diferença, se ela existir. Com 80%: se o `CLAUDE.md` ajuda mesmo, em 8 de cada 10 experimentos desse tamanho a gente veria.
- **Estimativa, não garantia:** a conta usa as taxas de hoje, que também têm margem de erro.

**Transição:** *"Para fechar, o que levar para casa."*

---

### Slide 23 — Leve para casa · [29:00–31:00]

**Diga:** *"Se vocês lembrarem de uma frase só: 'parece que melhorou' não é medida. Dá para medir o seu agente com tarefas e um teste automático, repetindo várias vezes, e comparar o antes e o depois. Três hábitos estão no slide: decida o que é dar certo antes de olhar o número; repita e olhe a margem de erro; toda falha vira tarefa nova. E mais dois: tarefas reais com teste escondido, e uma mudança por vez."*

**Diga:** *"O repositório roda o experimento de hoje com o login do Claude Code. São três comandos. E o ebook em PDF está dentro dele."*

**Faça:** deixar o slide parado enquanto as pessoas fotografam o QR. Conte até cinco antes de falar de novo. Abra para perguntas **neste slide** (~9 min, de 31:00 a 40:00).

**Desafio para quem levar o repo:** *"Troque o `CLAUDE.md` por uma skill ou um hook e rode de novo. Ou aponte as tarefas para o seu próprio projeto."*

---

### Slide 24 — Referências (apêndice)

Só mostre se alguém pedir as fontes. **Diga, se mostrar:** *"Todas as referências são públicas e estão no ebook com link. O SWE-bench está aí porque é o mesmo formato, em escala de pesquisa: issues reais do GitHub como tarefa para agente de código."* Depois, volte ao slide 23 (QR).

---

## A revelação em 3 versões

Escolha a versão no slide 18, olhando o título do relatório. As três contam a mesma história: **16 execuções de cada lado quase nunca bastam, e essa é a lição.** Use os números do relatório de hoje (ou do ensaio, no plano B); não arredonde nem "melhore" nenhum.

**Regra do veredito (para não errar ao vivo):**

- **"Com CLAUDE.md foi melhor" (B melhor):** a faixa de B fica toda acima da de A.

- **"Sem CLAUDE.md foi melhor" (A melhor):** a faixa de A fica toda acima da de B. É o **único** caso em que o portão barra (código 1).
- **"Ainda não dá para saber" (inconclusivo):** as faixas se cruzam.

## Versão 1 — B ganha

**Abertura:** *"O `CLAUDE.md` ganhou, e com faixas que não se encostam. Agora a pergunta que importa: ganhou onde?"*

**Leia:** tarefa por tarefa. O ganho está nas tarefas de regra da casa (03 a 07)? É o efeito esperado. Está também nas de controle (01, 02, 08)? Diga: *"isso me deixa desconfiado: o arquivo não deveria mudar nada ali. Pode ser acaso. Vale rodar de novo."*

**Lição:** *"Para aparecer com 16 execuções, a diferença precisou ser grande. Se o meu arquivo tivesse ajudado só um pouco, o relatório estaria dizendo 'ainda não dá para saber', e o efeito seria igualmente real. Uma diferença pequena, 62,5% contra 75%, pediria 212 execuções de cada lado. E ganhar uma vez não é ganhar sempre: o próximo passo é repetir outro dia."*

**Fechamento adaptado do slide 22:** *"Desta vez 16 bastaram, porque a diferença era grande. Na maioria das mudanças que vocês vão fazer, não vai ser. Agora vocês sabem calcular quanto rodar."*

## Versão 2 — Ainda não dá para saber (inconclusivo)

**Abertura:** *"Ainda não dá para saber. E eu não vou fingir que isso é empate."*

**Leia:** as duas linhas e as duas faixas, e aponte o trecho em que elas se cruzam. Depois, a frase do topo: "seriam umas [N] execuções de cada" (no ensaio, 27).

**Lição:** *"Pode ser que o `CLAUDE.md` ajude, e pode ser que não. Com 16 execuções de cada lado, o dado não deixa dizer. Se eu tivesse rodado uma vez só, como todo mundo faz, eu sairia daqui com uma certeza falsa. O relatório diz que eu precisaria de [N] execuções de cada lado. Esse número é o resultado mais útil de hoje."*

**Sobre o portão:** *"E o portão não ficou vermelho. 'Não sei ainda' não barra a mudança, porque falta de dado não é prova de piora."*

**Se B está na frente mas é inconclusivo:** *"B está na frente, mas 'na frente' com essas faixas é exatamente o 'parece que melhorou' do começo. A diferença é que agora eu sei o tamanho da minha dúvida."*

## Versão 3 — A ganha

**Abertura:** *"Sem o arquivo foi melhor. O meu `CLAUDE.md` atrapalhou, e o portão teria barrado a minha própria mudança."*

**Se quiser mostrar:** no terminal, o `pnpm eval` terminou com código 1, e o pnpm imprime uma mensagem de erro. *"Isso não é bug: é o portão funcionando."*

**Leia:** tarefa por tarefa, onde A ganhou. Hipóteses para falar em voz alta (sem afirmar nenhuma antes de abrir as execuções): uma instrução do arquivo que confunde o agente numa tarefa de controle; mais texto para ler levando a mais voltas e a estourar o limite; uma regra que empurra o agente a mexer fora do escopo.

**Lição:** *"Eu escrevi esse arquivo achando que ajudava. Se eu tivesse confiado no 'parece', ele estaria no repositório agora, piorando o agente de todo mundo. Medir serviu exatamente para isso. Mas são 16 execuções: dá para não fazer o merge, não para concluir que todo* `CLAUDE.md` *atrapalha. O próximo passo é abrir as execuções que falharam e transformar o que eu vir numa tarefa nova."*

### Casos de borda

- **Muitos erros do agente** (rede, cota, limite de tempo): diga *"olhem: em vez de 16, a variante X teve [n] execuções válidas. Erro do agente fica à parte, e a margem de erro ficou maior. É mais um motivo para não concluir nada com pouca amostra."* Se a maioria for erro, prefira o plano B.
- **Uma variante sem nenhuma execução válida:** o relatório mostra "Falta dado para comparar". Vá de plano B.

---

## Plano B: o run ao vivo falhou

**Quando usar:** no slide 18, se o `pnpm eval` não terminou, caiu (rede, login, cota), terminou com quase tudo erro do agente, ou o relatório mostra um número diferente de 16.

**Passo a passo:**

1. **Não mate o processo no palco** e não tente consertar.
2. No segundo terminal (já aberto na pasta do repo), rode `pnpm relatorio:planob`. Ele copia o relatório do ensaio (`resultados/ensaio/relatorio.html`) para `resultados/relatorio.html`, o arquivo que o slide mostra.
3. Volte um slide e avance para o 18 de novo: o slide recarrega e mostra o relatório do ensaio.
4. Siga o roteiro do slide 18 e a versão da revelação do ensaio ("Ainda não dá para saber"; você decorou na véspera).
5. **Se nem isso funcionar:** use a aba reserva com `resultados/ensaio/relatorio.html`, ou rode `pnpm relatorio:ensaio` e abra esse arquivo no navegador.

**O que dizer:**

> *"O run de hoje não terminou a tempo. Isso acontece com agente de verdade, e é por isso que existe um ensaio: o mesmo comando, as mesmas 32 execuções, gravadas antes da palestra. O relatório que eu vou mostrar é esse. E quem levar o repositório consegue gerar esse*

> *mesmo relatório sem internet e sem assinatura."*

Se o run caiu por erro do agente, acrescente: *"E é por isso que erro do agente é um rótulo próprio: se eu contasse isso como falha, o meu `CLAUDE.md` teria 'piorado' por causa do Wi-Fi."*

**Se o run terminar durante as perguntas:** o `pnpm eval` já terá gravado o relatório de hoje em `resultados/relatorio.html` (por cima do plano B). Dá para voltar ao slide 18 como bônus: *"o de hoje acabou de sair; vamos ver se concorda com o do ensaio."* Se não concordar, é mais uma prova da lição. Depois, volte ao slide 23 (QR).

---

## Perguntas prováveis

São ~9 min de perguntas (31:00–40:00), com o slide 23 (QR) na tela.

1. **"E o Codex? E o Gemini CLI?"** *"O método é o mesmo; só muda quem faz a tarefa. No repo, a parte que chama o agente fica isolada num lugar só, a interface `RodarAgente`. Para medir outro agente, você escreve outra versão dela, que chama a ferramenta dele. Tarefas, teste escondido e relatório continuam iguais. Está como desafio extra no repo. E não é para decidir qual ferramenta é melhor: é para você medir a configuração que você usa."*

2. **"IA como juiz serve para quê, então?"** *"Para o que não cabe num teste: clareza da documentação, descrição do PR, se a mensagem de erro é útil. A condição é medir o juiz também: dar a sua nota numa amostra e ver se ele concorda. E nunca usar juiz para o que um teste consegue conferir."*

3. **"Quantas execuções eu preciso?"** *"Depende do tamanho da diferença que você quer enxergar. Uma diferença grande, 37,5% contra 75%, pede umas 27 de cada lado. Uma diferença mais realista, 62,5% contra 75%, pede umas 212. O relatório do repo faz essa conta com os seus números e mostra no topo. Comece pequeno, veja a estimativa e decida se vale rodar mais."*

4. **"Isso não fica caro?"** *"O repo usa a cota da assinatura do Claude Code, com o login normal. O relatório mostra o custo estimado. Dá para economizar: os testes do próprio medidor rodam sem modelo e de graça, então rodam a cada commit; a eval com o agente de verdade roda só quando você muda alguma coisa no agente; e o `pnpm eval:rapido` roda uma tarefa só, para ver se está tudo de pé."* Não dê o custo do run de hoje sem ter o número do relatório na tela.

5. **"Preciso de chave de API?"** *"Não. O repo usa o login normal do Claude Code. E quem não tem assinatura consegue gerar o relatório do run gravado com `pnpm relatorio:ensaio`, sem internet."* Não recomende chave de API nem como alternativa.

6. **"E se o meu projeto não tem teste?"** *"Então o primeiro passo é escrever o teste da tarefa. Se você não consegue escrever um teste que diga se a tarefa foi cumprida, nem você sabe o que pediu. Comece com uma tarefa só, de um erro que o agente já cometeu."*

7. **"Quais foram os números do KTLO?"** *"São números internos que eu não posso compartilhar. O que eu posso passar são as lições e um método que vocês reproduzem no repo: lá está o run gravado do ensaio, e vocês podem rodar o de vocês."* Não dê faixa, ordem de grandeza nem "mais ou menos". Se insistirem: *"não posso mesmo; mas o método está todo no repo, e o número que importa para vocês é o do agente de vocês."*

8. **"Por que só duas repetições?"** *"Porque duas já mostram o ponto: dão 16 execuções de cada lado, e o relatório acabou de mostrar que é pouco. Para a palestra, eu quis um run curto (no ensaio, uns 3 minutos). No seu projeto, rode mais: é só mudar o* `repeat` *em* `evals/promptfooconfig.yaml`*."*

9. **"O agente não pode trapacear? Editar o teste?"** *"O teste escondido nunca entra na pasta do agente. A correção roda numa pasta nova, com o app intacto e só os arquivos que ele mudou, e o teste entra por último. Se ele mexer em arquivo fora dos permitidos, a execução conta como 'mexeu onde não devia'. E as minhas configurações pessoais não vazam para a cópia."*

10. **"Exigir faixas sem cruzar não é rigoroso demais?"** *"É, de propósito. Um teste estatístico clássico declararia vencedor antes. Para um portão automático, eu prefiro errar para o lado do 'não sei': um portão que grita cedo demais ensina o time a ignorá-lo. Se você quiser outra regra, ela fica num arquivo só, e dá para trocar."*

**Se o tempo apertar:** responda 1, 3 e 7 primeiro; são as que mais aparecem, e a 7 é a que exige mais cuidado.

---

## Glossário rápido

| Termo | Em uma frase |
|---|---|
| Agente de código | um modelo que lê, edita e roda código sozinho, usando ferramentas, até cumprir uma tarefa |
| `CLAUDE.md` | arquivo de texto com as regras do projeto, que o Claude Code lê no começo de toda conversa |
| Variante | cada versão comparada (A sem `CLAUDE.md`, B com) |
| Eval | teste automático de um sistema com IA: tarefas com um jeito automático de dizer se deu certo |
| Critério de feito | a regra objetiva que diz se a tarefa foi cumprida (aqui, um teste) |
| Teste escondido | o teste que decide a tarefa, que o agente não vê (a prova que o aluno não viu) |
| Escopo | os arquivos que a tarefa permite alterar; "fora do escopo" = mexeu onde não devia |

| Termo | Em uma frase |
|---|---|
| Tarefa de controle | caso em que a mudança não deveria fazer diferença |
| Token | pedacinho de texto que o modelo lê ou escreve |
| Turno | uma ida e volta entre o modelo e uma ferramenta |
| Headless | rodar o agente sem tela interativa (`claude -p`) |
| Sandbox | "caixa" do sistema operacional que limita o que o programa pode fazer |
| Provider | no promptfoo, o código que executa cada caso |
| promptfoo | ferramenta gratuita para rodar evals a partir de um arquivo de configuração |
| Diff | o que mudou nos arquivos em relação à cópia original |
| pass@k | passou em pelo menos uma de k tentativas |
| pass^k | passou em todas as k tentativas ("resolve sempre?") |
| Margem de erro | a faixa onde a taxa verdadeira provavelmente está (no jargão, intervalo de confiança) |
| Intervalo de Wilson | o jeito de calcular a margem de erro que o repo usa; funciona bem com poucas execuções |
| Poder | chance de o experimento enxergar uma diferença que existe |
| Inconclusivo | as faixas se cruzam: ainda não dá para dizer quem é melhor |
| LLM-as-judge | IA como juiz: um modelo dando nota ao trabalho de outro |
| Hook | script que roda antes ou depois de cada ação do agente e pode bloquear a ação |
| Skill | "manual" que o agente só abre quando a tarefa pede |
| CI | esteira que roda os testes a cada mudança no repositório |
| Portão de qualidade | regra automática que aprova ou barra uma mudança |
| KTLO | *keep the lights on*: manter funcionando o que já existe |
| Gabarito | o resultado esperado de cada caso, escrito por uma pessoa |
| Catraca | mecanismo que só gira para um lado: a cobertura da eval só cresce |